

Relatório de Segurança

Padrão OWASP

Projeto: trampoJá

Desenvolvedor: Matheus Barbosa Torres Gomes

Data: Junho/2026

Introdução

Este relatório apresenta uma análise de segurança do projeto trampoJá com base no padrão OWASP Top 10 (2021), que lista as dez categorias mais críticas de vulnerabilidades em aplicações web. O objetivo é identificar riscos existentes no código-fonte, documentar o que já está protegido e recomendar melhorias para versões futuras.

O trampoJá é um marketplace de serviços de curta duração voltado para a cidade de Araxá, MG. O app conecta clientes que precisam de serviços (jardinagem, encanamento, pintura, etc.) com trabalhadores locais. Possui versão web (React + Vite) e mobile (React Native + Expo), ambas utilizando Firebase Authentication para login.

Metodologia

A análise foi realizada através de revisão manual do código-fonte, examinando os arquivos trampo-ja.jsx (versão web) e App.js (versão Expo). Para cada item do OWASP Top 10, foram verificados os seguintes aspectos:

Primeiramente, foi analisada a arquitetura do sistema para entender como os dados fluem entre o frontend e o Firebase. Em seguida, cada fluxo de autenticação foi rastreado do input do usuário até a chamada ao Firebase Auth, verificando se há validações adequadas e tratamento de erros seguro.

As dependências do projeto foram verificadas quanto às suas versões e presença de vulnerabilidades conhecidas. Os formulários foram analisados quanto à sanitização de inputs e proteção contra injeção de código. Por fim, os testes automatizados do Playwright foram utilizados como evidência de que as validações funcionam corretamente na prática. Por fim, uma varredura final foi realizada pelo modelo de IA Claude da Anthropic.

Análise Detalhada

1. A01:2021 - Controle de Acesso Quebrado

Esta categoria trata de situações em que usuários conseguem acessar funcionalidades ou dados além do que deveriam ter permissão. É a vulnerabilidade mais comum em aplicações web segundo o OWASP.

Resolvido: No trampoJá, a proteção de rotas está implementada de forma funcional: a tela de login é o ponto de entrada obrigatório, e sem autenticação o usuário não consegue acessar o feed de serviços, o chat, as notificações ou qualquer outra funcionalidade do app. O estado de autenticação é gerenciado pelo React e pelo Firebase Auth, que controla a sessão do usuário.

No entanto: Entretanto, a separação entre perfis de cliente e profissional ainda está parcialmente implementada. O formulário de cadastro coleta o tipo de usuário, mas ambos os perfis veem as mesmas telas e têm acesso às mesmas funcionalidades. Em uma versão futura, seria importante restringir ações como "candidatar-se a um serviço" apenas para profissionais, e "publicar um serviço" apenas para clientes.

Recomendação: Implementar Firestore Security Rules com controle por uid e role, e diferenciar as permissões entre clientes e profissionais no frontend.

2. A02:2021 - Falhas Criptográficas

Esta categoria aborda a proteção inadequada de dados sensíveis, como senhas, tokens e informações pessoais, seja em trânsito ou em repouso.

Resolvido: As senhas dos usuários são gerenciadas inteiramente pelo Firebase Authentication, que utiliza algoritmos de hash seguros (bcrypt/scrypt) internamente. Em nenhum momento as senhas são armazenadas em texto plano ou manipuladas diretamente pelo código do app. A comunicação entre o app e o Firebase ocorre via HTTPS, que é o padrão do Firebase Hosting e das APIs do Firebase. Isso garante que dados sensíveis não trafeguem em texto plano na rede.

No entanto: O único ponto de atenção é que as chaves de configuração do Firebase (apiKey, authDomain, etc.) estão expostas diretamente no código-fonte. Embora essas chaves sejam públicas por design do Firebase (a proteção é feita via domínios autorizados e Security Rules), é uma boa prática movê-las para variáveis de ambiente em produção.

Recomendação: Mover o objeto FIREBASE_CONFIG para um arquivo .env e garantir que o .env esteja no .gitignore

3. A03:2021 - Injection

Injeção ocorre quando dados não confiáveis são enviados a um interpretador como parte de um comando ou query. Os tipos mais comuns são SQL injection, NoSQL injection e Cross-Site Scripting (XSS).

Resolvido: O trampoJá não utiliza banco de dados SQL, eliminando o risco de SQL injection. O Firebase Firestore, quando implementado, utiliza queries estruturadas que previnem NoSQL injection por design. Quanto ao XSS, o React oferece proteção nativa e o trampoJá não utiliza dangerouslySetInnerHTML em nenhum ponto, o que significa que mesmo que um usuário insira código malicioso em um campo de texto, ele será renderizado como texto puro e não executado.

No entanto: Os inputs dos formulários (título do serviço, descrição, mensagens do chat) não possuem sanitização adicional antes de serem processados. Embora o React proteja a renderização, quando esses dados forem persistidos no Firestore e eventualmente compartilhados entre usuários, seria prudente sanitizá-los.

Recomendação: Integrar uma biblioteca como DOMPurify para sanitizar inputs de texto antes de salvá-los no banco de dados.

4. A04:2021 - Design Inseguro

Design inseguro se refere a falhas na arquitetura da aplicação que deixam brechas para ataques, independentemente da implementação.

Resolvido: O sistema de validação do trampoJá é robusto. Os formulários de login e registro validam formato de e-mail via regex, comprimento mínimo de senha (6 caracteres), campos obrigatórios, e confirmação de senha no cadastro. Essas validações foram confirmadas pelos testes automatizados do Playwright (TC02, TC03, TC04, TC07). O Firebase Auth implementa rate limiting automaticamente, bloqueando tentativas de login após múltiplas falhas consecutivas o que previne ataques de força bruta sem necessidade de implementação adicional.

Um Problema: Não há separação entre ambientes de desenvolvimento, staging e produção. O mesmo projeto Firebase é usado em todos os contextos.

Recomendação: Criar projetos Firebase separados para desenvolvimento e produção, evitando que testes afetem dados reais.

5. A05:2021 - Configuração Incorreta de Segurança

Configuração incorreta acontece quando a aplicação possui configurações padrão inseguras, recursos desnecessários habilitados, ou permissões excessivas.

Resolvido: O Firebase Authentication está configurado corretamente com apenas dois métodos de login habilitados: Email/Senha e Google Sign-In. A lista de domínios autorizados restringe de onde o login pode ser feito, e o CORS é gerenciado automaticamente pelo Firebase.

6. A06:2021 - Componentes Vulneráveis e Desatualizados

O uso de bibliotecas e frameworks desatualizados pode introduzir vulnerabilidades conhecidas na aplicação.

Resolvido: As principais dependências do trampoJá estão atualizadas: Firebase SDK na versão 10.12.0, React 18+ tanto no Vite quanto no Expo, e Leaflet 1.9.4 para mapas na versão web. Essas são versões recentes e não possuem vulnerabilidades críticas conhecidas.

No entanto: É necessário executar “npm audit” periodicamente para verificar se novas vulnerabilidades foram descobertas nas dependências do projeto. A execução do npm audit durante o desenvolvimento revelou 3 vulnerabilidades (1 low, 1 moderate, 1 high), que devem ser corrigidas com npm audit fix.

Recomendação: *Executar “npm audit” fix antes de cada release e manter uma rotina de atualização das dependências.*

7. A07:2021 - Falhas de Identificação e Autenticação

Esta categoria cobre falhas no processo de verificação de identidade do usuário, incluindo senhas fracas, sessões inseguras e ausência de proteções adicionais.

Resolvido: O trampoJá utiliza Firebase Authentication, que é um serviço maduro e amplamente utilizado. O fluxo de login por e-mail/senha inclui validação de formato, comprimento mínimo, e tratamento de erros para credenciais inválidas, muitas tentativas, e problemas de rede. O Google Sign-In está disponível na versão web como alternativa ao login manual.

Um Problema: Não há funcionalidade de recuperação de senha. O Firebase oferece o método sendPasswordResetEmail() que envia um link de redefinição por e-mail, mas ainda não foi integrado ao app.

Recomendação: *Implementar a funcionalidade de recuperação de senha utilizando sendPasswordResetEmail do Firebase, e avaliar a implementação de MFA para contas de profissionais.*

8. A08:2021 - Falhas de Integridade de Software e Dados

Esta categoria trata de situações em que código ou dados podem ser modificados sem verificação adequada.

Resolvido: O npm utiliza o arquivo package-lock.json que contém checksums de todas as dependências, garantindo que os pacotes instalados não foram adulterados. Isso protege contra ataques de supply chain na cadeia de dependências.

9. A09:2021 - Falhas de Log e Monitoramento de Segurança

Sem logs e monitoramento adequados, é impossível detectar ataques em andamento, investigar incidentes ou identificar padrões suspeitos.

Resolvido: O Firebase Authentication mantém automaticamente um log de todos os eventos de autenticação (login, registro, falhas) acessível pelo Firebase Console. Isso oferece visibilidade básica sobre a atividade de autenticação.

Um Problema: Não há serviço de monitoramento de erros integrado ao app. Exceções e falhas no frontend não são registradas em nenhum serviço externo, o que significa que bugs e potenciais ataques podem passar despercebidos. Também não há alertas configurados para atividades suspeitas como múltiplas tentativas de login falhas de um mesmo IP.

Recomendação: *Integrar Firebase Crashlytics para a versão mobile e Sentry para a versão web, configurando alertas para eventos críticos de segurança.*

10. A10:2021 - Server-Side Request Forgery (SSRF)

SSRF ocorre quando um atacante consegue fazer o servidor da aplicação realizar requisições para destinos não autorizados, potencialmente acessando serviços internos.

Resolvido: O trampoJá é uma aplicação frontend-only que não possui servidor backend próprio. Todas as operações são realizadas diretamente entre o navegador/app e os serviços do Firebase. Como não há servidor intermediário que processe URLs fornecidas pelo usuário, o risco de SSRF é inexistente na arquitetura atual.

Conclusão

A análise do trampoJá com base no OWASP Top 10 revela que o projeto possui uma base de segurança sólida, principalmente graças à utilização do Firebase Authentication como serviço de autenticação. Dos dez itens analisados, 3 estão adequadamente resolvidos, 4 estão parcialmente implementados, e 3 itens estão pendentes de implementação. O item mais crítico pendente é a configuração de regras de acesso no Firestore quando o backend for implementado.

As principais melhorias recomendadas para as próximas versões são: implementar Firestore Security Rules, mover credenciais para variáveis de ambiente, adicionar recuperação de senha, e integrar monitoramento de erros.

Plano de Ação Priorizado

Com base na análise, as seguintes ações são recomendadas em ordem de prioridade:

1. Implementar Firestore Security Rules com controle por uid e role.
2. Mover credenciais Firebase para variáveis de ambiente (.env).
3. Adicionar funcionalidade "Esqueci minha senha" com sendPasswordResetEmail.
4. Sanitizar inputs com DOMPurify antes de persistir dados.
5. Configurar CI/CD com GitHub Actions para verificação automática de build.
6. Integrar monitoramento de erros com Sentry (web) e Crashlytics (mobile).
7. Remover console.log em builds de produção.
8. Separar ambientes de desenvolvimento, staging e produção.